If you use the cookie session store, this would apply to the `session` and `flash` hash as well.

7 Rendering XML and JSON data

ActionController makes it extremely easy to render `XML` or `JSON` data. If you've generated a controller using scaffolding, it would look something like this:

```
class UsersController < ApplicationController
  def index
    @users = User.all
    respond_to do |format|
      format.html # index.html.erb
      format.xml  { render xml: @users}
      format.json { render json: @users}
    end
  end
end
```

You may notice in the above code that we're using `render xml: @users`, not `render xml: @users.to_xml`. If the object is not a String, then Rails will automatically invoke `to_xml` for us.

8 Filters

Filters are methods that are run before, after or "around" a controller action.

Filters are inherited, so if you set a filter on `ApplicationController`, it will be run on every controller in your application.
"Before" filters may halt the request cycle. A common "before" filter is one which requires that a user is logged in for an action to be run. You can define the filter method this way:

```
class ApplicationController < ActionController::Base
  before_action :require_login

  private

  def require_login
    unless logged_in?
      flash[:error] = "You must be logged in to access this section"
      redirect_to new_login_url # halts request cycle
    end
  end
end
```

The method simply stores an error message in the flash and redirects to the login form if the user is not logged in. If a "before" filter renders or redirects, the action will not run. If there are additional filters scheduled to run after that filter, they are also cancelled.

In this example the filter is added to `ApplicationController` and thus all controllers in the application inherit it. This will make everything in the application require the user to be logged in in order to use it. For obvious reasons (the user wouldn't be able to log in in the first place!), not all controllers or actions should require this. You can prevent this filter from running before particular actions with `skip_before_action`:

```
class LoginsController < ApplicationController
  skip_before_action :require_login, only: [:new, :create]
end
```

Now, the `LoginsController`'s `new` and `create` actions will work as before without requiring the user to be logged in. The `:only` option is used to only skip this filter for these actions, and there is also an `:except` option which works the other way. These options can be used when adding filters too, so you can add a filter which only runs for selected actions in the first place.